

```
(venv_Q) D:\All Virtualenv\quantum>pip install qiskit
Collecting qiskit
  Downloading qiskit-2.1.2-cp39-abi3-win_amd64.whl.metadata (13 kB)
Collecting rustworkx<=0.15.0 (from qiskit)
  Downloading rustworkx-0.17.1-cp39-abi3-win_amd64.whl.metadata (10 kB)
Collecting numpy<3,>=1.17 (from qiskit)
  Using cached numpy-2.2.6-cp310-cp310-win_amd64.whl.metadata (60 kB)
Collecting scipy<=1.5 (from qiskit)
  Downloading scipy-1.15.3-cp310-cp310-win_amd64.whl.metadata (60 kB)
Collecting dill<=0.3 (from qiskit)
  Downloading dill-0.4.0-py3-none-any.whl.metadata (10 kB)
Collecting stevedore>=3.0.0 (from qiskit)
  Downloading stevedore-5.5.0-py3-none-any.whl.metadata (2.2 kB)
Collecting typing_extensions (from qiskit)
  Downloading typing_extensions-4.15.0-py3-none-any.whl.metadata (3.3 kB)
```

Figure 1: Installation of *qiskit* module

```
(venv_Q) D:\All Virtualenv\quantum>pip install qiskit-aer
Collecting qiskit-aer
  Downloading qiskit-aer-0.17.1-cp310-cp310-win_amd64.whl.metadata (1.2 kB)
Requirement already satisfied: qiskit>=1.1.0 in d:\all virtual (1.2)
Requirement already satisfied: numpy>=1.16.3 in d:\all virtual (2.6)
Requirement already satisfied: scipy>=1.0 in d:\all virtualenv (3)
Collecting psutil>=5 (from qiskit-aer)
  Using cached psutil-7.0.0-cp37-abi3-win_amd64.whl.metadata (6.5 kB)
Collecting python-dateutil>=2.8.0 (from qiskit-aer)
  Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl (291 kB)
Collecting six>=1.5 (from python-dateutil>=2.8.0->qiskit-aer)
  Using cached six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
Requirement already satisfied: rustworkx>=0.15.0 in d:\all virtualenv (0.17.1)
Requirement already satisfied: dill>=0.3 in d:\all virtualenv (0.4.0)
Requirement already satisfied: stevedore>=3.0.0 in d:\all virtualenv (5.5.0)
Requirement already satisfied: typing-extensions in d:\all virtualenv (4.15.0)
```

Figure 2: *qiskit_aer* module installation

```
from qiskit import QuantumCircuit
from qiskit_aer import Aer

# Create quantum circuit
qc = QuantumCircuit(1, 1)
qc.h(0)
qc.measure(0, 0)

# Use Aer simulator
simulator = Aer.get_backend("qasm_simulator")
job = simulator.run(qc, shots=1000)
result = job.result()
counts = result.get_counts()

print("Result:", counts)
```

Figure 3: Hadamard gate implementation

The output is:

```
(venv_Q) D:\All Virtualenv\quantum>python q1.py
Result: {'0': 509, '1': 491}
```

Here's the explanation of this example.

- QuantumCircuit lets us build quantum circuits. Aer is used to simulate the quantum computer. We make a circuit with 1 quantum bit (qubit) and 1 classical bit to store measurement results.
- `qc.h(0)` applies a Hadamard gate on the qubit. This puts the qubit in superposition (both 0 and 1).
- `qc.measure(0, 0)` measures the qubit and stores the result (0 or 1) into the classical bit.
- We use Aer's *qasm_simulator*, which mimics a quantum computer for running our circuit.
- We run the circuit 1000 times (`shots=1000`) to get probabilities of results.
- `result.get_counts()` shows how many times we got 0 and how many times we got 1.
- Out of 1000 runs, ~509 times the qubit collapsed to 0 and ~491 times it collapsed to 1.
- This is very close to 50–50, because the Hadamard gate ensures equal probability.
- This shows a nearly equal chance of 0 and 1 because of superposition.

Quantum computing is still in the early stages of development but has a huge future.

With Python and libraries like Qiskit, anyone can learn the basics of quantum programming, even on a conventional laptop. It is a great way to understand how the next generation of computers will work. **END** 🐧

By: Mohit Raj

The author is a CEH and ECSA, and has worked at IBM, Teramatrix and Sapient. He is the author of the books titled 'Python Penetration Testing Essential', 'Python: Penetration Testing For Developers', and 'Learn Python in 7 Days'.